Architectures Logicielles et Matérielles
Cours 05 – Gestion des interruptions

Bruno Ferres
basé sur un cours de Philippe Waille

10 septembre 2026

Contenu de la séance

Objectifs

- ajouter un mécanisme d'interruption rudimentaire au processeur "jouet" vu en L2 (INF401)
- illustrer la programmation d'interruption en `C`

Conventions et concepts

Utilisation de noms génériques pour :

- les registres : `slr` (ou `lr_irq`), `spsr`
- les instructions : `bl`, `accept`, `ignore`, `rti`

Limites

Ne présente pas divers mécanismes avancés des processeurs modernes, dont :

- modes d'exécution : restreint *vs* privilégié
- bancs de registres spécifiques aux modes d'exécution
- traitants d'interruption eux-mêmes interruptibles (cascade)

Autre vision d'un CPU : un interpréteur d'instruction

une instruction $\triangleq$ une sémantique $\triangleq$ **plusieurs micro-actions**

Rôle du processeur — *plusieurs étapes, entre 2 et. . .25 ?*

1. **Fetch** : récupère la prochaine instruction à exécuter
2. **Decode** : décide quelle instruction exécuter (quel chemin dans l'automate), les opérandes, . . .
3. **Execute** : fait les calculs nécessaire pour réaliser l'effet de l'instruction (notamment via l'**U**nité **A**rithmétique et **L**ogique)
4. **Memory** : réalise les accès mémoires nécessaires (`ldr/str`)
5. **Write Back** : met à jour l'état architectural (*i.e.* les registres) en fonction de l'effet de l'instruction

↪ que faire quand on a fini ? mettre à jour `pc` et passer à la suite !

↪ **tout ça implémenté en matériel**

Un jeu d'instruction simple pour mieux comprendre

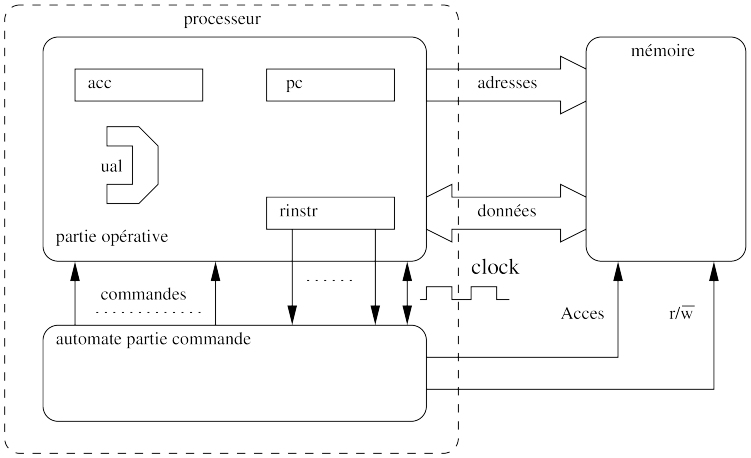
Processeur "jouet" vu en INF401

- seulement **deux registres** accessibles au programmeur : `pc` et `acc`
- un registre (non accessible) pour l'instruction courante : `rinstr`
- un registre pour accéder au bus d'adresse (en écriture) : `ma`
- un registre pour le bus de données (lecture/écriture) : `md`

Jeu d'instruction — *Instruction Set Architecture* (**ISA**)

Format instruction	mots	sémantique
`clear`	1	Acc ← 0
`load` `vimm`	2	Acc ← vimm
`jump` `adr`	2	pc ← adr
`store` `adr`	2	Mem[adr] ← Acc
`add` `adr`	2	Acc ← Acc + Mem[adr]

Synoptique du processeur



L'horloge du processeur

Un processeur **synchrone**

- le processeur est piloté par un signal d'horloge périodique
  - ↪ souvent appelé `clock`
  - ↪ on parle de **cycle d'horloge** (front montant → front montant)
- la partie commande (PC) est un **automate câblé** qui effectue une transition d'état à chaque cycle d'horloge
- la partie opérative (PO) est un circuit de calcul, qui peut effectuer un calcul par cycle d'horloge

  ↪ *en tout cas, en première approximation*

Ordres de grandeur

- sur les processeurs modernes, $f \in [1,5]$ GHertz (période $\leq$ 1ns)
- le microcontrôleur STM32F407 fonctionne à 168 MHz

Squelette de l'algorithme d'interprétation des instructions

Un fonctionnement câblé matériellement dans le processeur

```
1: pc ← 0                                          ▷ initialisation
2: while true do                                   ▷ pourquoi ?
3:     rinstr ← Mem[pc] ; pc ← pc +1               ▷ Fetch
4:     switch rinstr do                            ▷ Decode
5:         case clear: ...                         ▷ sémantique du clear
6:         case load: ...                          ▷ sémantique du load
7: end while                                       ▷ Execute/Memory/WriteBack fusionnés
```

se traduit par le graphe de l'automate (Partie Contrôle)

Sémantique des instructions et actions associées

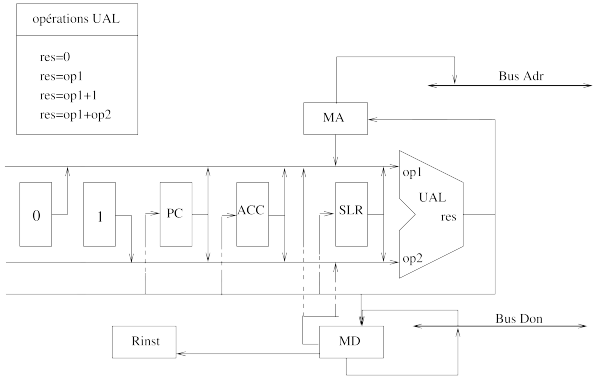
Sémantique $\triangleq$ effet sur l'architecture $\triangleq$ suite de micro-actions

Instruction	Sémantique	Actions associées à l'instruction
`clear`	acc ← 0	acc ← 0
`load vim`	acc ← vimm	acc ← Mem[pc] ; pc ← pc + 1
`jump adr`	pc ← adr	pc ← Mem[pc] ; pc ← pc + 1
`store adr`	Mem[adr] ← acc	Mem[Mem[pc]] ← acc ; pc ← pc + 1
`add adr`	acc ← acc + Mem[adr]	acc ← acc + Mem[Mem[pc]] ; pc ← pc + 1

Attention : processeur "jouet" à instructions de tailles variables

Après l'étape `Fetch`, l'opérande (`vim` ou `adr`) est dans Mem[pc]
↪ il faut donc lire Mem[pc] avant de l'utiliser comme opérande

Des contraintes imposées par la Partie Opérative (PO)

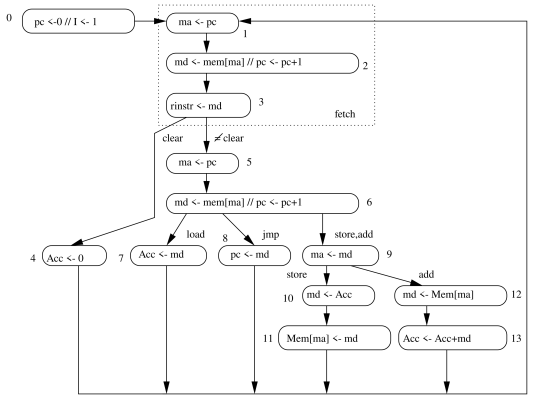


Seuls accès mémoire : `md` ← Mem[`ma`] et Mem[`ma`] ← `md`

Exemple pour le *Fetch*

`rinstr` ← Mem[`pc`] ⇒ {`ma` ← `pc` ; `md` ← *Mem*[`ma`] ; `rinstr` ← `md`}
(1) (2) (3)

Automate d'interprétation de la Partie Contrôle (PC)



transition de chaque dernier état → *Fetch*

Hypothèse

1 accès mémoire = 1 cycle

Durée du *load* : 1 tour de boucle en 6 états ⇒ **6 cycles**

Durée du *store* : 1 tour de boucle en 8 états ⇒ **8 cycles**

Registre d'état du processeur

Current Program Status Register CPSR

31	30	29	28	27 – 8	7	6 – 0
N	Z	C	V	réservés	I	réservés

↪ mis à jour par certaines opérations arith. ou logiques (ex : `cmp`, `tst`)

Utilisation

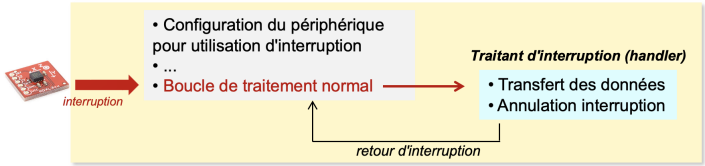
Permet d'implémenter des **ruptures de séquences (conditionnelles)**

- `bne finSi` : saute à `finSi` ssi Z
- `bal debutBoucle` : saute à `debutBoucle` **inconditionnellement**
- `ble finSi` : saute à `finSi` ssi $Z \vee (N \wedge \bar{V}) \vee (\bar{N} \wedge V)$

Rappels du cours 04 : mécanisme d'interruption (1/2)

C'est quoi, une interruption ?

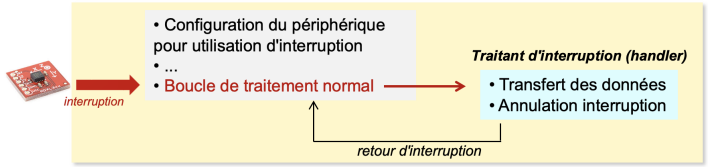
- permet de forcer **matériellement** l'exécution d'une (courte) routine **traitant** l'interruption ⇝ *interrupt handler/ interrupt service routine*
- en réponse à un évènement généré à l'extérieur du processeur
  ↪ par exemple, un périphérique (clavier, ...), un timer, ...
- **matérialisé** par un signal électrique booléen de requête
  ↪ nom typique : `IRQ` (**I**nterrupt **ReQ**uest)
- seulement si le bit **I** (**I***nhibit Interruption*) du registre d'état `CPSR` est à 0



Rappels du cours 04 : mécanisme d'interruption (2/2)

Interruption et programme : que faire au retour du traitant ?

l'exécution du programme se poursuit comme s'il n'avait pas été interrompu
↪ mécanisme transparent pour l'application
↪ l'exécution du programme interrompu n'est que légérement ralenti



Instant de prise en compte de IRQ

Quand peut on "partir" en interruption ?

Rappel : le CPU exécute en boucle {Fetch-Decode-Execute-Mem-WriteBack}

Le départ en interruption s'insère entre :

- la fin d'exécution (WriteBack) de l'instruction (machine !) courante
- le début d'exécution (Fetch) de l'instruction suivante

Une prise en compte non instantanée

Si IRQ passe à 1 pendant le Fetch ou l'Execute d'une instruction
↪ IRQ pris en compte juste avant le retour à Fetch pour l'instruction suivante

Activation et neutralisation du mécanisme d'interruption

Des instructions dédiées

- au démarrage du processeur, mécanisme désactivé : $I = 1$
- deux instructions spéciales (noms génériques) :
  - accept ($I \leftarrow 0$) : démasquer les interruptions
    ↪ autorise la prise en compte du signal IRQ
  - ignore ($I \leftarrow 1$) : masquer les interruptions
    ↪ interdit la prise en compte du signal IRQ

Comment programmer ?

- pas de syntaxe C associée : utilisation directe de l'instruction assembleur
- peut aussi se faire via
  {mov r0, cpsr ; modifier bit I dans r0 ; mov cpsr, r0}

Traitants d'interruption et fonctions

Les traitants d'interruptions sont des *routines* (blocs de code) exécutées quand l'interruption survient.
↪ de loin, on dirait des fonctions...
↪ mais il y a quelques différences majeures à comprendre !

Gestion des appels de fonction

Rappel : appel de fonctions "ordinaires" — ARM

- appel **explicite** à une fonction/procédure `func`
  ↪ à la compilation, on SAIT quand sera appelée la fonction
  ↪ instruction `bl func` (*Branch and Link*) dans l'appelante
- sémantique de `bl` :
  - **sauvegarder** l'adresse de retour : `lr` ← `pc`
  - appel de la fonction (saut) : `pc` ← `pc` + Δ

Retour de fonction

- l'adresse de retour dans `lr` contient l'instruction qui suit le `bl`
- la fonction appelée termine par un **branchement retour** : `bx lr`
  ↪ (presque) équivalent à `mov pc, lr`

Problème de la gestion des registres

le banc de registre est **partagé** entre appelante et appelée

Convention d'appel et responsabilités

La fonction appelée peut modifier des registres :

- pc (`r15`) : forcément modifié → forcément sauvegardé (`lr`)
- `CPSR` (`Z`, `C`, `N` et `V`) : pas sauvegardé — pourquoi ?
- registres généraux (`r0` à `r14`) – dépend de la convention d'appel

  Pour ARM (**AAPCS** – *Arm Architecture Procedure Call Standard*) :
  - `r0` à `r3` sauvegardés par l'appelante (dans la pile)
  - `r4` à `r14` sauvegardés par l'appelée (dans la pile)
    ↪ empilé (prologue) puis dépilé (épilogue)

Attention aux sauvegardes des registres

```
void main() {                        main: push  {r0,r5,lr}
  char c='x'; // stocké dans r5             mov   r5,#'x'
  f();                                      bl    f
  putchar(c);                               mov   r0,r5
                                            bl    putchar
}                                           pop  {r0,r5,lr}
                                            bx   lr

void f() {                             f: push  {r0,r5}   @ 1
  char r5='z'; // stocké dans r5            mov   r5,#'z'
  putchar(r5);                              mov   r0,r5
                                            bl    putchar
}                                           pop   {r0,r5}   @ 2
                                            bx    lr
```

Avec push/pop dans `f` (`@1` et `@2`) : affichage `zx`

Sans push/pop dans `f` (sans `@1` et `@2`) : affichage `zz`

Différences entre traitants d'interruption et fonctions

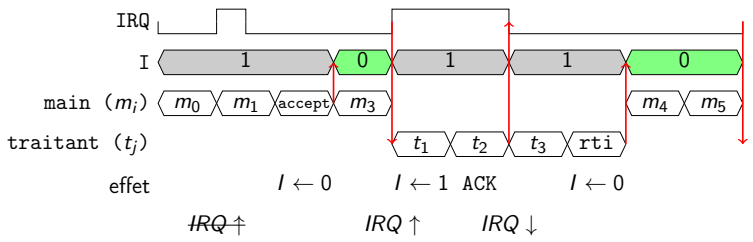
Un départ différent

- une fonction est **appelée explicitement** dans le programme
  ↪ le programmeur doit **respecter la convention de sauvegarde**
- un traitant d'interruption peut dérouter le code depuis (presque) partout
  ↪ le programmeur ne peut pas prévoir la sauvegarde dans le programme
  ↪ c'est au traitant de sauvegarder !

Un retour pas pareil

- une fonction rend simplement la main à l'appelante (`bx lr`)
- le traitant doit restaurer certaines choses, car l'interruption **ne peut pas être prévue dans l'appelante !**
  ↪ puis retour avec une instruction spécifique (ici, `rti`)

Déroulement temporel d'une interruption



État du processeur

- l'état du CPU au début de m_4 doit être identique à celui en fin de m_3
  ↪ le `handler` ne doit pas modifier les registres
  ↪ il doit sauvegarder tous les registres utilisés !
- `I` == 1 pendant le `handler`
  ↪ pas de départ possible en interruption !

Branchement **câblé** pour appeler le traitant

Départ en interruption ≠ appel de fonction

- l'appel du traitant est **câblé** : **déclenché par un signal booléen** `IRQ`
- pourquoi pas de `bl traitant` dans le `main` interrompu ?
  - branchement câblé (matériel)
  - traitant potentiellement indépendant du `main`
    ↪ mise à jour de l'horloge pendant un calcul
  - l'interruption peut arriver à **n'importe quel moment** !

Quelle fonction appeler : adresse du `handler`

- l'adresse du `handler` doit être **définie par une convention**
- une adresse spéciale (`0x0018` en ARM32) définit l'adresse du handler
  ↪ **vecteur d'interruption** (≠ `handler` !)

Les deux formes de vecteur d'interruption

Choisir `pc` lors du départ déclenché par `IRQ`

- approche typiquement RISC (*e.g.* ARM 32 bits) :
  `pc` ← `vecteur_interruption`
  ↪ le vecteur contient une **instruction de branchement** (`bl`) vers le `handler`
- approche typiquement CISC (mais aussi celle du ARM CorTeX) :
  `pc` ← Mem[`vecteur_interruption`]
  ↪ le vecteur contient **l'adresse du handler**

Appel de traitant : sauvegarde des registres

Sauvegarde minimale **câblée**

- `pc` dans `slr` (ou `lr_irq`)
  ↪ pour savoir où revenir après l'interruption
- `cpsr` (registre d'état) dans `spsr`
  ↪ le bit `I` de `spsr` est forcément à 0 (pourquoi ?)

Registres généraux

Pour les registres généraux modifiés dans le corps du traitant :

- la sauvegarde (en pile) dans le prologue du traitant ;
- la restauration (depuis la pile) dans l'épilogue du traitant

sont écrites par le programmeur du traitant.

Appel de traitant : masquage et séquence de départ

Le mécanisme d'interruption doit être désactivé pendant l'exécution du traitant

↪ rappel : on ne gère pas les interruptions en cascade ici !

Séquence de départ – **déclenchée par** `IRQ`

1. `slr` ← `pc`
2. `spsr` ← `cpsr`
3. bit `I` dans `cpsr` ← 1
4. `pc` ← `vecteur_interruption` (ou `pc` ← Mem[`vecteur_interruption`])

Appel de traitant : retour

Une instruction de retour différente

- le traitant se termine par une instruction `rti` (ReTour d'Interruption)
- différente d'une instruction de retour ordinaire (`mov pc, lr` ou `bx lr`)
  - ↪ adresse de retour stockée dans `slr` et non `lr`
  - ↪ il faut aussi restaurer `cpsr`

Sémantique de `rti`

1. `cpsr` ← `spsr`
2. démasquer les interruptions : bit `I` de `cpsr` ← 0
   - ↪ effet de bord de l'action précédente
3. `pc` ← `slr`

en ARM 32 bits, `rti` se fait via `movS pc, lr`

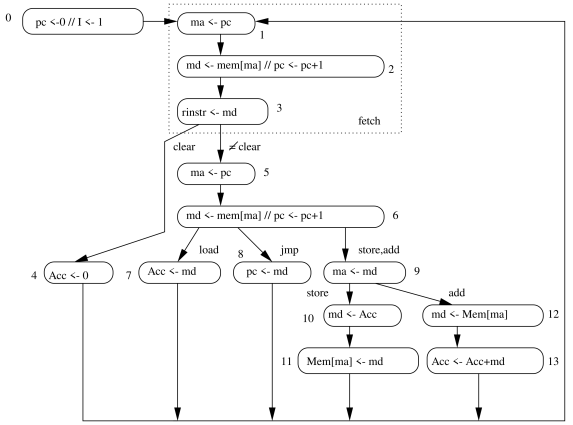
Rappel : Automate d'interprétation **sans interruption**

Squelette de l'agorithme **câblé dans le processeur**

```
1: pc ← 0                                          ▷ initialisation
2: while true do
3:     rinstr ← Mem[pc] ; pc ← pc +1               ▷ Fetch
4:     exécuter la sémantique selon rinstr         ▷ Dec/Exe/Mem/WB
5: end while
```

Dans le graphe, transition du **dernier état** de chaque instruction vers l'étape `Fetch`

Rappel : Automate de commande **sans interruption**



transition vers `Fetch` depuis chaque dernier état

Automate d'interprétation **avec interruptions**

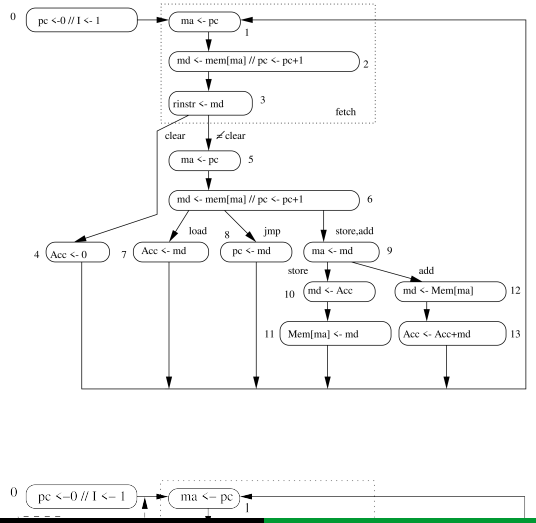
Squelette de l'agorithme **câblé dans le processeur**

```
 1: pc ← 0 ; I ← 1                          ▷ initialisation
 2: while true do
 3:     rinstr ← Mem[pc] ; pc ← pc +1       ▷ Fetch
 4:     exécuter la sémantique selon rinstr ▷ Dec/Exe/Mem/WB
 5:     if IRQ and Ī then                   ▷ départ en interruption
 6:         slr ← pc
 7:         spsr ← cpsr
 8:         I ← 1
 9:         pc ← vecteur_irq
10:     end if
11: end while
```

Dans le graphe, transition du **dernier état** de chaque instruction :

- vers l'étape `Fetch` si $\overline{IRQ \wedge \bar{I}} = \overline{IRQ} \vee I$
- vers la séquence de départ en interruption si $IRQ \wedge \bar{I}$ et après ?

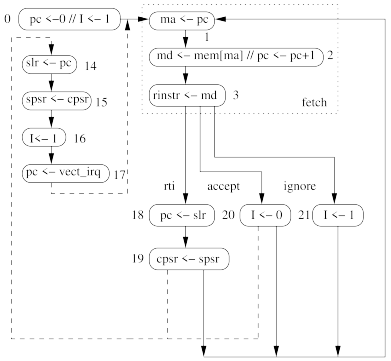
Automate de commande **avec interruptions**



transition depuis chaque dernier état vers :

- `Fetch` si $\overline{IRQ} \vee I$
- départ en interruption si $IRQ \wedge \bar{I}$

Ajout des instructions `rti`, `accept`, `ignore`



ajout des trois instructions à l'automate

`rti` est une **instruction ordinaire**

- gérée comme les autres
- utilisée en fin de traitant

Pour résumer

- départ en interrution *ssi* $IRQ \wedge \bar{I}$
- retour via `rti`

Utilisation de code C "ordinaire" dans un traitant

Problématique pour écrire un `handler` directement en C

Une fonction C "ordinaire" ne peut pas être utilisée directement :

- elle se termine par `bx lr` au lieu `rti`
  ↪ ne restaure pas `cpsr`, ~~`pc ← lr`~~ et non `pc ← slr`
- elle ne sauvegarde pas tous les registres modifiés dans son corps
  ↪ dépend de la convention d'appel !

Solution

Écrire un `handler` en langage d'assemblage qui :

- effectue les sauvegardes de registres nécessaires
- appelle la fonction C "normalement" (via `bl` ou `blx`)
- se termine par `rti`

`handler` écrit par le programmeur *"bare metal"* ou fourni dans le noyau de l'OS

Ceci **ne s'applique pas** au processeur CorTeX de la carte Discovery.

Application de la solution : handler C+asm

```
traitant_irq:   @ empiler les registres modifiables
                @ par la fonction
                push {r0-r3,ip,lr}
                @ appel normal de la fonction
                bl fonction
                @ restaurer les registres
                pop {r0-r3,ip,lr}
                @ retour du traitant via rti
                rti

// fonction C ‘‘ordinaire’’
void traitant_C (void) {
    *port_sortie = *port_entree;
}
```

Quelle garantie doit-on avoir sur `traitant_C` ?

Un mécanisme particulier

Utilisation de traitants en C

Le mécanisme d'interruption des processeur CorTeX est spécialement conçu pour utiliser des fonctions C ordinaires (terminées par `bx lr`) comme traitants.
↪ simple à utiliser, mais **complexe à comprendre**

Principe de fonctionnement

Câblage (en **matériel**) :

- de la sauvegarde des registres
  ↪ remplace notre traitant `traitant_irq` en langage d'assemblage
- du choix de l'instruction de retour : `bx lr` *vs* `rti`

Mécanisme de départ en interruption **câblé**

Traitement câblé lors d'une interruption `IRQ`

1. empiler `r0, r1, r2, r3, ip/12` et `lr/r14`
   ↪ pourquoi ces registres là, au juste ? toujours la convention d'appel !
2. empiler `pc`
   ↪ sauvegarde dans la pile au lieu de `slr`
3. empiler `cpsr`
   ↪ sauvegarde dans la pile au lieu de `spsr`
4. `lr` ← une adresse de retour illégale ($\in [0xFFFFFFF0; 0xFFFFFFFF]$)
5. `pc` ← `*vecteur_interruption`

Retour d'interruption câblé

Modification de la sémantique de `bx lr`

Lors de l'exécution de l'instruction `bx lr`, test du contenu de `lr/r14` :

- si c'est une adresse légale
  ↪ il s'agit d'une fonction appelée “normalement”
  ↪ `pc` ← `lr`
- si c'est une adresse illégale, `bx lr` $\simeq$ `rti` :
  1. dépiler `cpsr`
  2. dépiler `pc` (effectue le branchement retour)
  3. dépiler `r0, r1, r2, r3, ip/r12` et `lr/14`

Exercice

Supposons qu'on dispose encore une fois d'un thermomètre comme périphérique, qui, cette fois ci, génère une interruption quand la température dépasse le seuil de 30°C.
De plus, on dispose toujours de la macro `THERM_PORT_PTR` qui permet de récupérer la température T (en Kelvin) :

```
#define THERM_PORT_PTR ((volatile uint32_t) *) 0x5000
```

Écrire le traitant d'interruption en `C` qui est appelé lorsque l'interruption est traitée par le processeur, et qui :

- allume la LED bleue si $T \in [30°C; 35°C]$
- allume la LED rouge si $T > 35°C$

On considérera deux cas (c'est à dire, deux traitants) :

1. le cas où la lecture du périphérique (`THERM_PORT_PTR`) **acquitte automatiquement** l'interruption
2. le cas où l'interruption doit être **acquittée explicitement** via un appel à la fonction `ack_therm_read()`

Sources d'interruptions multiples : balayage logiciel

Comment gérer plusieurs sources d'interruption ?

- on fait un `OU` logique ($\vee$) de tous les signaux de requête
  $\hookrightarrow IRQ = \sum_x IRQ_x$
- dans le `handler` :
  - on regarde **séquentiellement** chaque IRQ_x autorisée
  - si plusieurs $IRQ_x = 1$:
    $\hookrightarrow$ on gère la première interruption
    $\hookrightarrow$ on repart immédiatement en interruption après chaque retour
    $\hookrightarrow$ jusqu'à avoir géré tous les IRQ_x

Notion de priorité

L'ordre de parcours des IRQ_x dépend de la **priorité relative** des sources
$\hookrightarrow$ le parcours prend du temps

Sources d'interruptions multiples : table de vecteurs

Comment éviter le balayage logiciel ?

- doter chaque IRQ_x de son propre signal et de son propre vecteur
  $\hookrightarrow$ table de vecteurs d'interruption
- priorité des sources définie par un **circuit encodeur de priorité**

Cas des processeurs ARM CorTeX

la priorité de chaque source est configurable dans le `NVIC`
$\hookrightarrow$ Nested Vectorized Interrupt Controler